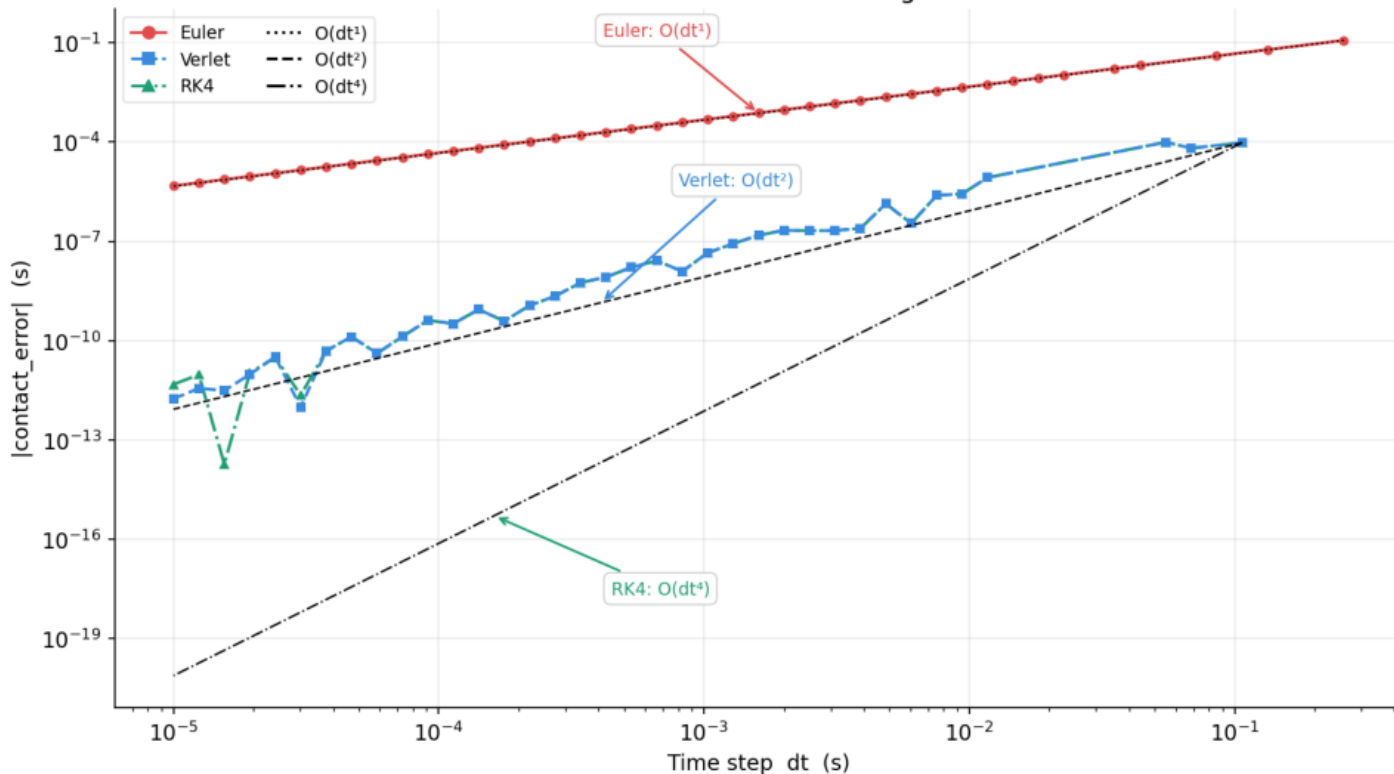


Contact time error vs dt — convergence order

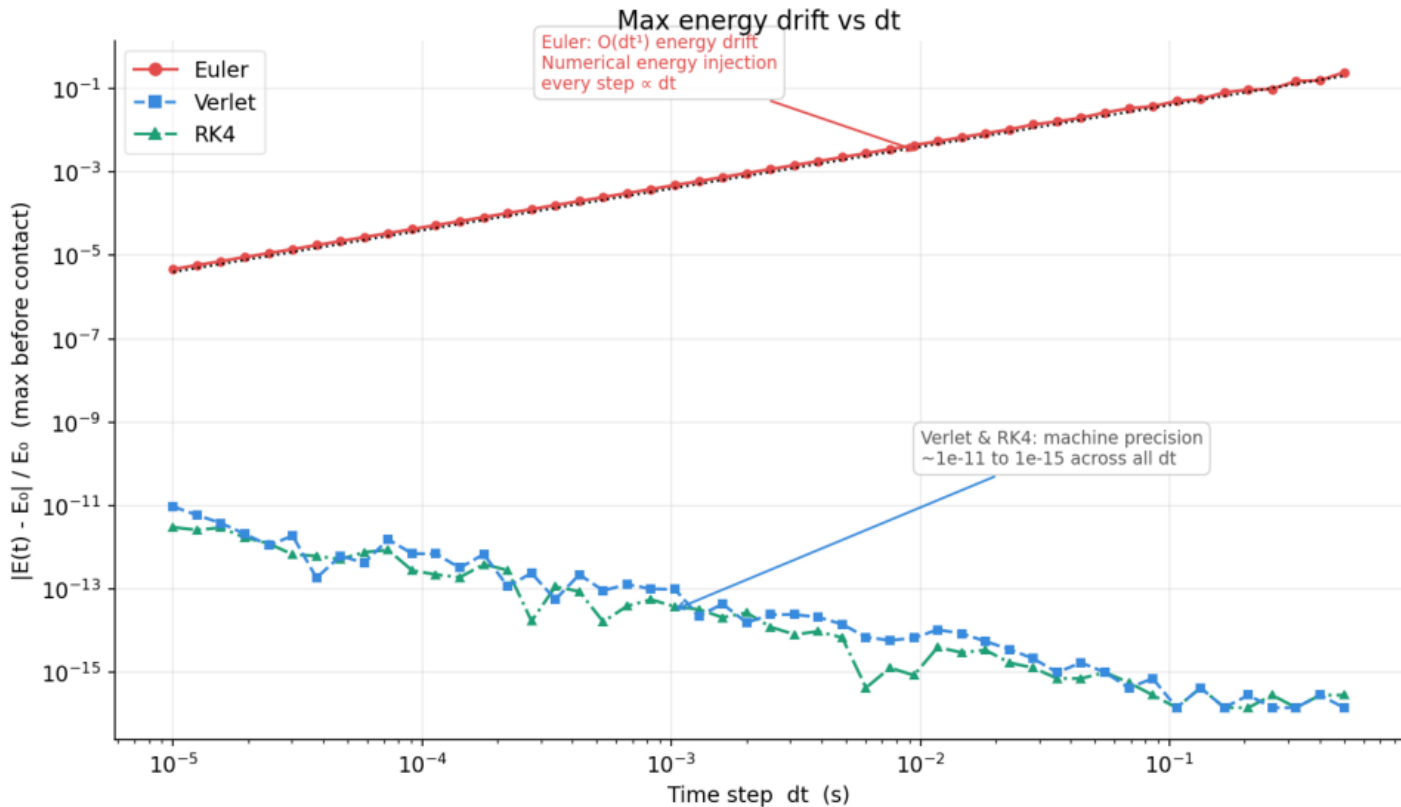


CONVERGENCE ORDER

Euler $\rightarrow O(dt^1)$
 Perfect first-order slope across the entire dt range. Textbook result for explicit Euler.

Verlet $\rightarrow O(dt^2)$
 Clean second-order slope with fluctuations.

RK4 $\rightarrow O(dt^2)$ instead of $O(dt^4)$
 Free fall is a degree-2 polynomial in time. RK4 integrates polynomials up to degree 4 exactly $\rightarrow$ only rounding error remains. $O(dt^4)$ slope cannot appear on this test case.
 $\rightarrow$ Use a pendulum to see it.



ENERGY CONSERVATION

Euler

Non-conservative integrator. Injects artificial energy at every step. Drift scales as $O(dt^1)$ – perfectly linear on this log-log plot. 6-10 orders of magnitude worse than Verlet/RK4. → Never use for long sims.

Verlet

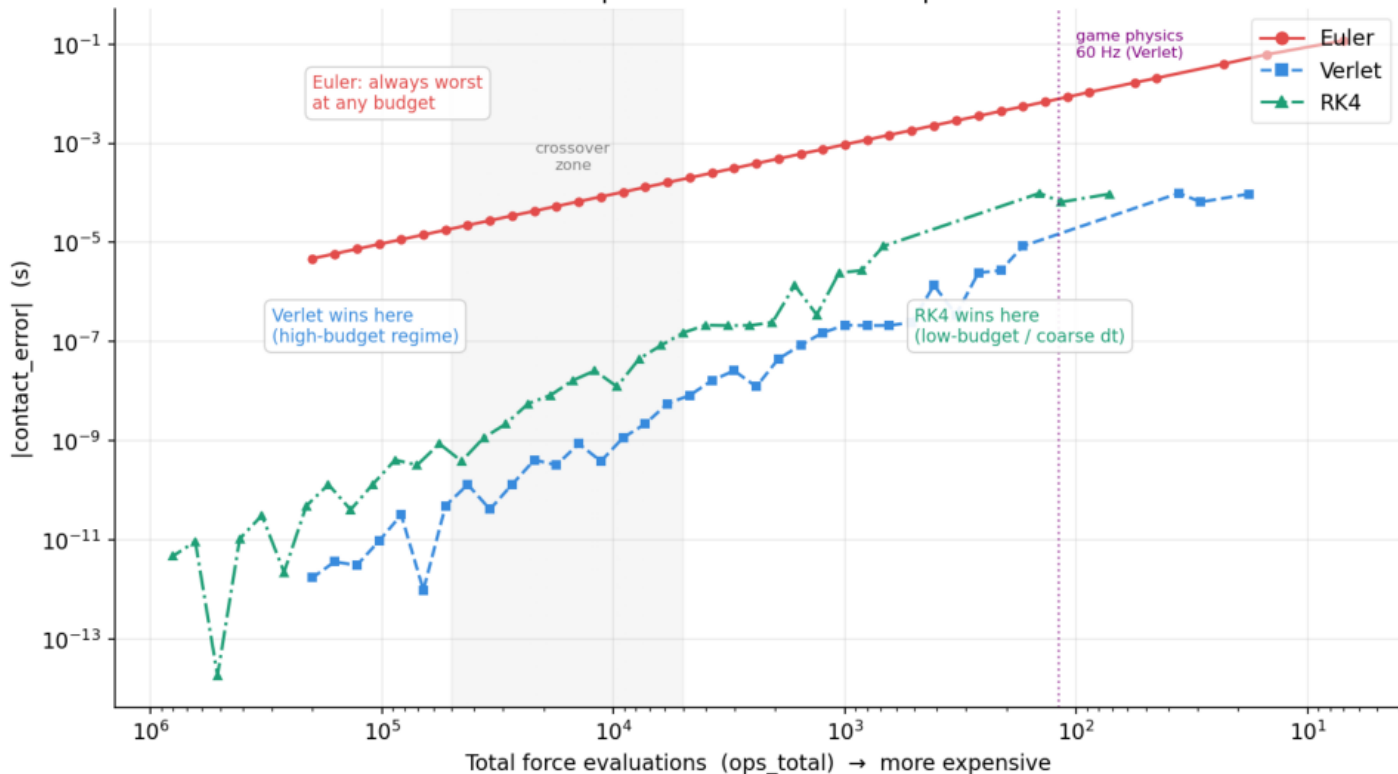
Symplectic integrator: conserves a modified Hamiltonian exactly. Drift is bounded (no secular growth), sitting at the double-precision floor $\sim 10^{-14}$.

RK4

Not symplectic, but so accurate on this polynomial problem that drift is also at machine epsilon. On a nonlinear system, RK4 would show slow secular drift unlike Verlet.

Note: measurement stops at contact to avoid the penetration spike polluting the max.

Cost vs precision — iso-cost comparison



COST vs PRECISION

X-axis: total force evaluations
(Euler=1/step, Verlet=1/step, RK4=4/step)
← left = more expensive

Euler

Always dominated. At any given budget, Verlet or RK4 will give a much smaller error.

Verlet vs RK4

Crossover $\sim 1e4$ ops:

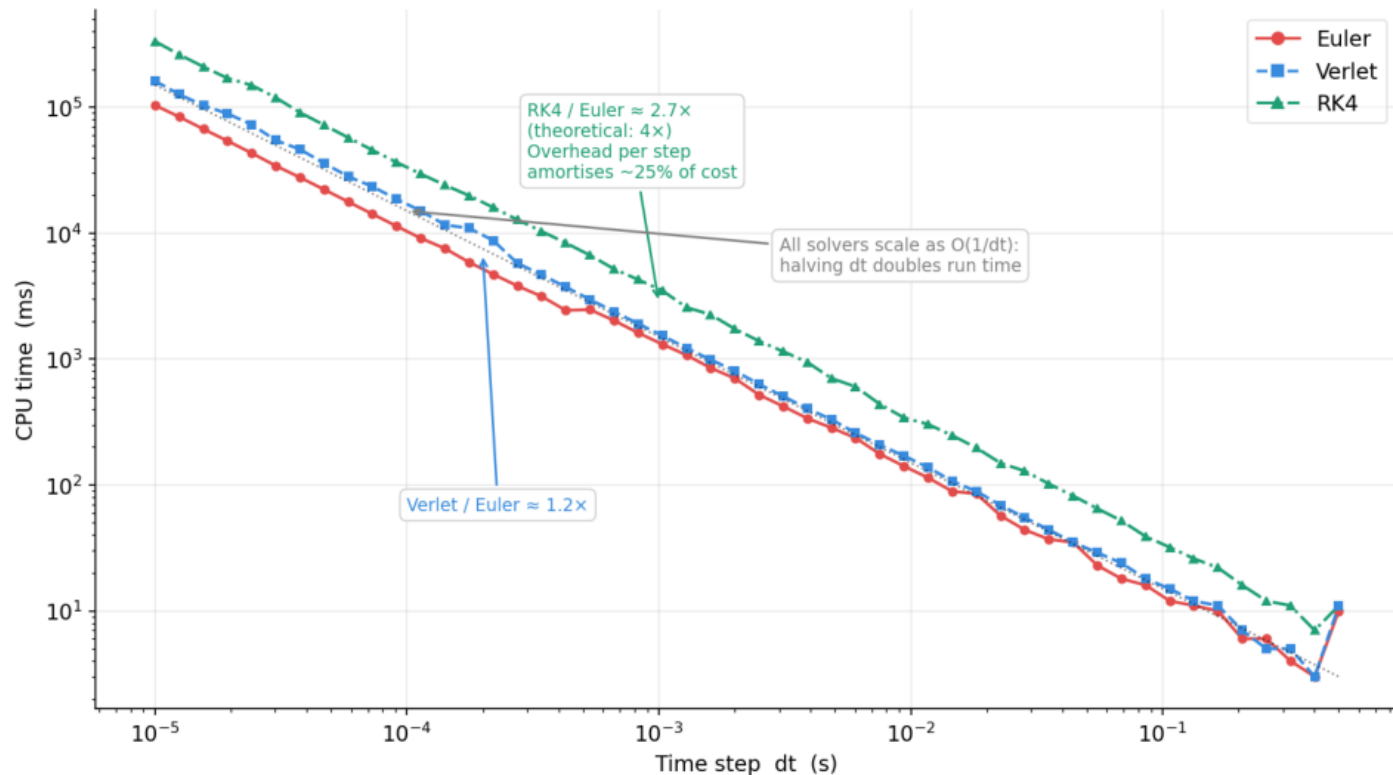
- Left (high cost): RK4 wins
- Right (low cost): Verlet wins

For a game engine at 60 Hz

(dt $\sim 1/60$ s $\rightarrow$ ~ 120 ops for Verlet vs ~ 480 for RK4 at same dt):
 $\rightarrow$ Verlet is the clear winner.

RK4 only makes sense if you need sub- $1e-9$ s precision, which is never relevant in real-time physics.

CPU time vs dt



CPU TIME

All three solvers scale as $O(1/dt)$: parallel lines of slope -1 on this log-log plot. Halving dt doubles CPU time.

Measured cost ratios

(vs Euler = $1\times$):

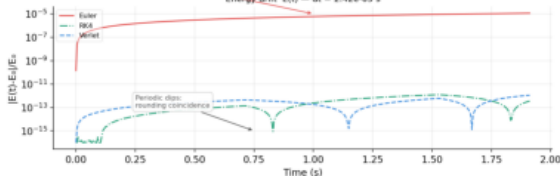
Verlet : $\sim 1.2\times$

RK4 : $\sim 2.7\times$

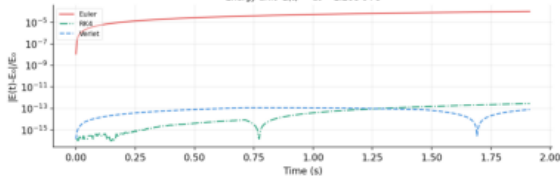
RK4 is theoretically $4\times$ more expensive (4 force evals/step) but measures at only $\sim 3\times$. The gap comes from per-step overhead (collision check, memory, object lookup) shared by all solvers regardless of integration method.

Verlet's $1.5\times$ overhead vs Euler is the cost of the extra x_{prev} storage and the central difference velocity update.

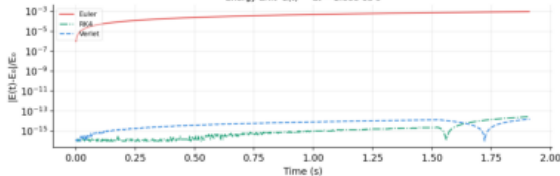
Euler monotonic energy injection
Energy drift: $E(t) - dt = 2.42e-05$ s



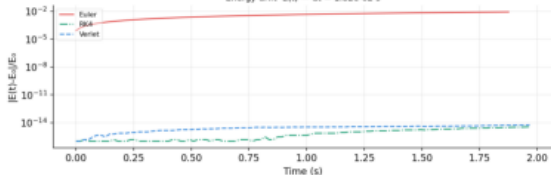
Energy drift: $E(t) - dt = 2.20e-04$ s



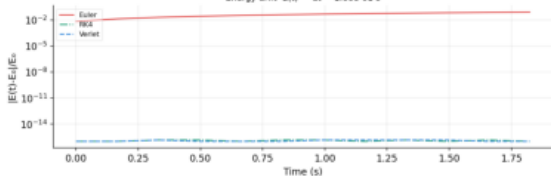
Energy drift: $E(t) - dt = 2.00e-03$ s



Energy drift: $E(t) - dt = 1.82e-02$ s



Energy drift: $E(t) - dt = 1.66e-01$ s



ENERGY DRIFT $E(t)$

Euler (red)
Monotonic growth from $t=0$.
Pure artificial energy injection
every step. Rate = dt
• $dt=2.4e-5$: drift $\sim 1e-5$
• $dt=1.6e-1$: drift $\sim 1e-2$
No oscillation, no spikes –
clean first-order dissipation.

Verlet & RK4 (blue/green)
Flat at $1e-14$ to $1e-15$ for all
 dt values – machine precision.

Periodic dips (small dt panels)
At $t = 0.75, 1.1, 1.6$ s the
sphere passes through a state
where E_k and E_p round-trip
through identical float values
– near-zero relative drift.
Physically meaningless coincidence,
not a conservation event.

Contact spike ($t = 1.9$ s)
Velocity changes discontinuously
at impact. Energy is briefly
inconsistent before the next
integration step. Expected and
not a bug. Measurement now
stops before contact so this
no longer pollutes plot 2.

As dt grows (bottom panels)
power dips visible (fewer steps)
but the qualitative picture is
the same: Euler drifts, Verlet
and RK4 conserve.